

Chapter 4

Proposed Methodology

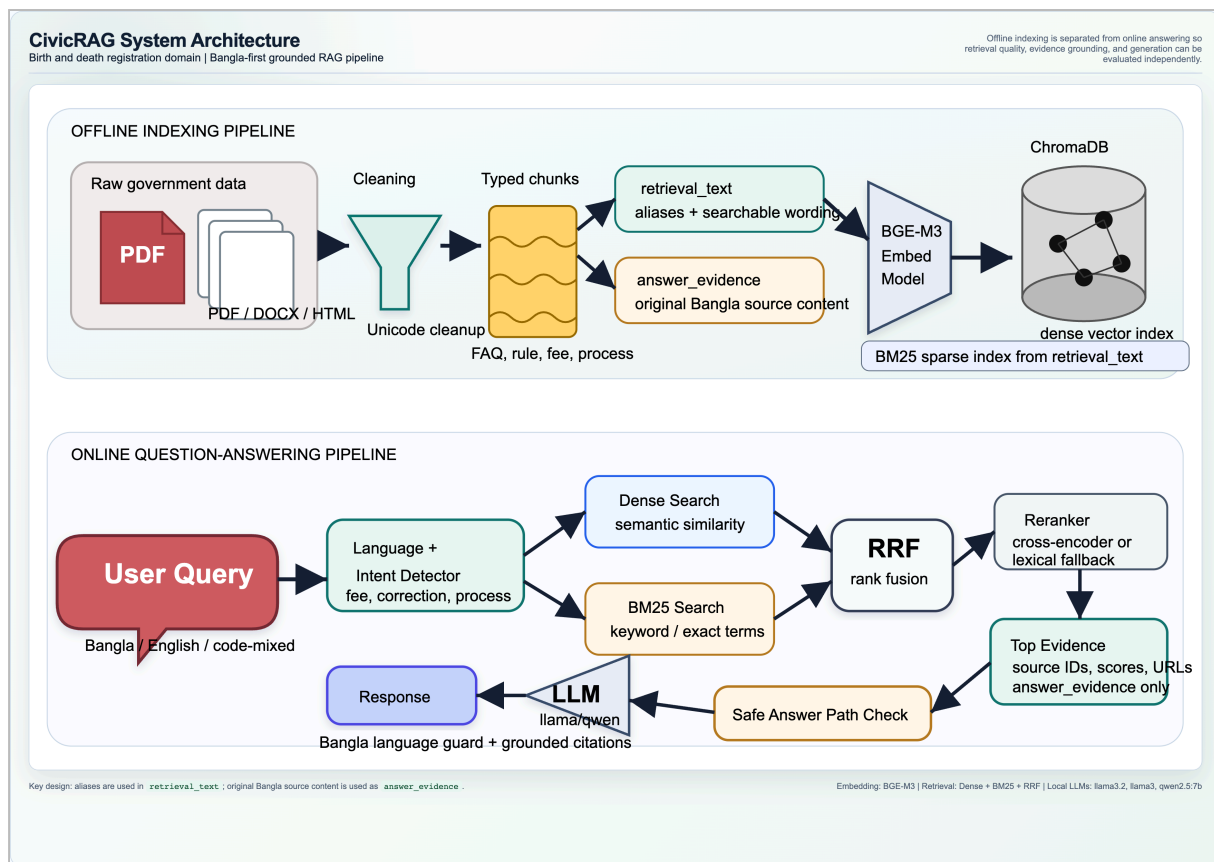
Design Process or Methodology Overview

Phase 2 of this thesis focuses on transforming the high-level idea of Civic.ai into a concrete, testable system design. In Phase 1, the work was framed as a citizen-centric AI-powered government service assistant. During Phase 2, the design was refined into an evidence-grounded Retrieval-Augmented Generation (RAG) system for Bangladeshi government-service question answering. The current implementation scope is limited to the birth and death registration domain so that the design can be evaluated carefully before adding additional domains such as passport, BRTA, or other civic services.

The central design requirement is factual correctness. A government-service assistant should not behave like a general creative chatbot. It must retrieve relevant evidence from trusted documents, generate answers from that evidence, expose source information, and reduce unsupported claims. For this reason, Civic.ai is designed using the RAG paradigm, where a retriever first selects relevant document chunks and a generator then produces an answer grounded in the retrieved evidence [1]. A recent bilingual Bangla-English RAG system, NextRAG, also demonstrates the importance of hybrid retrieval, preprocessing, reranking, and language-aware response generation in a Bangladeshi information-access setting [2]. However, Civic.ai does not claim to reproduce that system directly; instead, it applies related RAG principles to a different domain, namely public government-service guidance.

The design process follows five major stages:

1. Collection of raw government-service documents from birth and death registration sources.
2. Manual and semi-manual cleaning to remove web noise, duplicate navigation text, OCR artifacts, and formatting inconsistencies.
3. Document-type-aware chunking for FAQ, rule, fee-table, correction-process, and application-process documents.
4. Indexing with both dense multilingual embeddings and sparse lexical retrieval.
5. Retrieval, fusion, reranking, answer generation, and evaluation using curated Bangla scenario-based questions.



Proposed CivicRAG architecture for the birth and death registration domain. The offline indexing pipeline is separated from the online question-answering pipeline so that retrieval quality and generation quality can be evaluated independently.

Figure 4.1 shows the overall system architecture. The offline pipeline processes raw government documents, cleans and chunks the data, creates two text versions per chunk, embeds the searchable text, and stores it in a vector database. The online pipeline accepts a user query, detects language and civic intent, retrieves candidate chunks using dense and sparse retrieval, fuses ranked results, reranks evidence, and finally generates a grounded response using either controlled answer paths or a local LLM.

Preliminary Design or Design (Model) Specification

Several alternative design solutions were considered and simulated. The purpose of this comparison was not only to choose one final system but also to justify why the selected design is appropriate for a factual Bangla government-service assistant.

Alternative Design Solutions

Alternative design solutions considered for Civic.ai in Phase 2.

Design Alternative	Description	Reason for Evaluation
Dense-only RAG	Uses multilingual dense embeddings and vector similarity search.	Strong candidate for paraphrased Bangla queries where exact words may differ from source documents.
BM25-only RAG	Uses sparse lexical retrieval based on term matching.	Useful for exact government terms, legal rule numbers, fee terms, and service names [3].
Hybrid RAG	Combines dense retrieval and BM25 using rank fusion.	Provides a more robust architecture for future multi-domain expansion where both semantic and exact matching may matter.
CivicRAG	Adds metadata, source URLs, language and intent detection, reranking, safe answer paths, and local LLM generation.	Selected design because it supports explainability, factual grounding, and controlled answers for high-risk civic-service questions.

The dense retrieval component is motivated by prior work on dense passage retrieval for open-domain question answering [4] and sentence-level semantic representation learning [5]. In the current implementation, the embedding model is BAAI/bge-m3 because it is designed for multilingual, multi-functionality, and multi-granularity retrieval [6]. This is important for Civic.ai because the knowledge base is Bangla-heavy while user queries may appear in Bangla, English, or code-mixed form.

The sparse retrieval component uses BM25, a strong probabilistic lexical retrieval baseline [3]. BM25 is especially useful when a query contains exact service words, fee-related words, rule references, or official terminology. The simplified BM25 scoring function can be represented as:

$$\text{BM25}(q,d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t,d)(k_1 + 1)}{f(t,d) + k_1(1 - b + b \cdot |d| / \text{avgdl})}$$

Here, q is the query, d is a document chunk, $f(t,d)$ is the frequency of term t in chunk d , $|d|$ is the chunk length, and avgdl is the average chunk length. The parameters k_1 and b control

term-frequency saturation and length normalization [3].

To combine dense and sparse retrieval results, the system uses Reciprocal Rank Fusion (RRF). RRF combines multiple ranked lists without requiring the raw scores of different retrievers to be directly comparable [7]. In the current system, a weighted form of RRF is used:

$$\text{RRF}(d) = \sum_{r \in R} \frac{w_r}{k + \text{rank}_r(d)}$$

where d is a candidate chunk, R is the set of retrievers, w_r is the retriever weight, k is the RRF constant, and $\text{rank}_r(d)$ is the rank of chunk d under retriever r . The original RRF formulation is unweighted, but the weighted version is useful for Phase 2 sensitivity analysis because it allows the dense and BM25 contributions to be tuned.

After fusion, a reranking stage is used to improve the order of candidate chunks. Cross-encoder reranking is a common approach for passage reranking because it jointly scores the query and candidate passage [8]. The current Civic.ai implementation supports cross-encoder reranking when the local model is available and uses deterministic lexical/domain-aware fallback reranking for local reproducibility.

Simulation and Verification of Alternative Designs

A curated Bangla evaluation set was created to simulate real citizen questions. The current evaluation set contains 58 paraphrased questions grouped around 19 civic-service problems. These questions cover application processes, correction cases, lost certificate cases, fees, legal conditions, online visibility problems, and document requirements.

The retrieval alternatives were evaluated using ranking metrics commonly used in information retrieval, including Hit@k/Recall@k, Mean Reciprocal Rank (MRR), normalized Discounted Cumulative Gain (nDCG), and Context Precision [9, 10, 11]. The main retrieval results are shown in Table 4.2.

Retrieval variant comparison on the 58-question Bangla evaluation set.

Method	Hit@1	Hit@5	MRR	nDCG@5
BM25-only	0.5345	0.7586	0.6132	0.6112
Dense-only	0.7931	0.9828	0.8707	0.8693
Hybrid RRF	0.7414	0.9483	0.8193	0.8139

The result shows that dense-only retrieval currently performs best for the single-domain birth and death registration evaluation set. This is reasonable because many evaluation questions are paraphrased Bangla questions, where semantic similarity is more important than exact word overlap. However, hybrid retrieval remains part of the selected design because future multi-domain expansion may increase lexical ambiguity. For example, different government services may contain semantically similar instructions but domain-specific terms such as passport, driving license, birth registration, death registration, fee, correction, or application. In such settings, sparse retrieval can help preserve exact domain signals.

The following metrics were used:

$$\text{Recall@k} = \frac{\text{Number of queries where a relevant chunk appears in top k}}{\text{Total number of queries}}$$

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

MRR measures how early the first relevant chunk appears in the ranked list [10, 9]. For ranking quality with graded relevance, nDCG is used:

$$\text{DCG@k} = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}$$

$$\text{nDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}$$

where rel_i is the relevance score of the result at rank i , and IDCG@k is the ideal DCG at rank k [11].

The generation-side design was also examined through context-size testing. The goal was to determine how many retrieved chunks should be passed to the answer generator. Table 4.3 shows the trade-off between evidence coverage and context precision.

Generation context top-k coverage.

k	Expected Source in Context	Context Precision@k
1	0.7414	0.7414
3	0.8793	0.3621
5	0.9483	0.2448
8	0.9828	0.1616

The results show that larger context sizes improve the chance of including the expected evidence but also introduce more irrelevant chunks. The current selected value is k=3 for generation because it is conservative and reduces noise. However, k=5 is a candidate for future testing because it improves expected-source coverage.

Data Collection -(If Applicable)

The current dataset focuses on Bangladeshi birth and death registration services. The sources include government-service web pages, frequently asked questions, legal rules, fee information, application process documents, correction guidance, and scanned or converted documents related to birth and death registration.

Four major raw data types were considered:

- Bijoy-encoded PDF files.
- Converted DOCX files.
- Unicode HTML pages from government web portals.
- Scanned PDF or OCR-affected documents.

The current Phase 2 implementation primarily indexes manually reviewed and cleaned Markdown/JSON data. Raw scanned and converted files are retained for auditability, but only cleaned and structured content is used for the active vector database. This decision was made to reduce garbage-in-garbage-out errors in retrieval.

Data Cleaning

Government-service documents are noisy and heterogeneous. HTML pages often contain menus, accessibility controls, repeated footers, news blocks, lists of unrelated services, and navigation text. PDF and DOCX files may contain broken formatting, repeated headings, encoding issues, and OCR noise. Because RAG quality depends heavily on the quality of

retrieved evidence, data cleaning is treated as a core design step rather than a minor preprocessing task.

The cleaning process includes:

- Removing repeated website menus, footer blocks, accessibility controls, and unrelated service lists.
- Converting or replacing non-Unicode/Bijoy text where possible.
- Separating FAQ entries into individual question-answer units.
- Preserving legal and procedural wording where exact phrasing matters.
- Removing OCR fragments, empty headings, and duplicate content.
- Keeping official source URLs as metadata for explainability.

The purpose of cleaning is to prevent irrelevant text from being embedded and retrieved. If noisy navigation text is stored in the vector database, the retriever may return chunks that match common words but do not answer the citizen's question.

Data Transformation

After cleaning, the data is transformed into retrieval-friendly chunks. A single chunking method is not suitable for all document types. Therefore, Civic.ai uses document-type-aware chunking.

Chunking strategy by document type.

Document Type	Chunking Strategy	Reason
FAQ JSON	One question-answer pair per chunk.	FAQ answers are naturally self-contained and should remain intact.
Heading-based Markdown	Section and subsection based chunking.	Preserves procedural structure and improves evidence interpretability.
Fee tables	Row-level chunks with fee category metadata.	Fee questions often require precise row-level retrieval.
Legal/rule documents	Rule or paragraph-group based chunks.	Legal evidence should not be mixed with unrelated sections.
Correction/application process documents	Step or condition based chunks.	User questions often refer to specific scenarios or procedural steps.

Each chunk contains two separate text fields:

- **retrieval_text:** used for searching. It may include aliases, normalized terms, and query-friendly wording.
- **answer_evidence:** used for generation. It preserves original source content and is passed to the LLM for grounding.

This separation is important because aliases can improve retrieval, but the generator should not be grounded on artificial aliases. Therefore, the LLM receives original evidence text rather than expanded retrieval text.

Data Integration

All domain-specific resources are organized under one domain folder. The current domain is:

```
\begin{center} domains/birth_death_registration/ \end{center}
```

This folder contains raw documents, cleaned Markdown/JSON files, generated chunks, ChromaDB vector storage, configuration files, and evaluation datasets. This design supports future multi-domain expansion without creating separate repositories. Future domains such as

passport or BRTA can be added as new domain folders with their own data and configuration files.

Data Reduction

In this project, data reduction does not mean numerical dimensionality reduction. Instead, it means reducing irrelevant document noise before indexing and reducing retrieved candidate chunks before generation. The major reduction decisions are:

- Removing duplicate and irrelevant government-website boilerplate.
- Skipping duplicate FAQ content where the same answer appears in multiple sources.
- Splitting long documents into focused evidence units.
- Using top-k retrieval to avoid sending too much context to the generator.
- Applying reranking and intent-based boosting to prioritize the most relevant evidence.

This is necessary because long or noisy context can confuse LLM generation. In a civic-service system, adding more context is not always better; the system should provide enough evidence to answer the question but not so much that unrelated chunks influence the final response.

Summary of Preprocessed Data

The final preprocessed knowledge base contains Bangla-heavy birth and death registration chunks. The major chunk categories are FAQ, legal rules, fee rows, correction process, application process, portal guidance, and general service information. Each chunk contains metadata such as source ID, document type, section title, category, source URL, and evidence text.

For Phase 2 verification, a 58-question Bangla paraphrase evaluation set was created from 19 grouped civic-service problems. These questions are designed to test whether the system can retrieve the same expected evidence when a citizen asks the same problem in different wording.

Implementation of Selected Design

The selected design is CivicRAG, a modular local RAG system for government-service question answering. The implementation consists of six main modules:

1. **Data processing module:** reads cleaned Markdown and JSON files and converts them into typed chunks.
2. **Embedding module:** embeds `retrieval_text` using BAAI/bge-m3 [6].

3. **Indexing module:** stores dense vectors in ChromaDB and builds a BM25 sparse index.
4. **Retrieval module:** retrieves candidates using dense search, BM25 search, or hybrid RRF search.
5. **Reranking and safety module:** reranks candidates and checks whether a controlled answer path is available.
6. **Generation module:** produces a final answer using local LLMs when a controlled answer is not available.

The current local LLM backends are llama3.2, llama3, and qwen2.5:7b. Llama-based models are used as local open-weight baselines [12], while Qwen2.5 is included as an alternative multilingual local model [13]. The default generation temperature is set to 0.05 because factual government-service QA requires low randomness and high consistency.

For high-confidence civic intents such as fee questions, lost certificate questions, correction questions, and application-process questions, the system can use controlled answer paths. These paths reduce unnecessary LLM improvisation by producing answers directly from selected evidence patterns. If no controlled path is available, retrieved evidence is passed to the local LLM with a strict grounding prompt.

Generation quality is planned to be evaluated using answer correctness, answer relevance, faithfulness, and qualitative human review. RAG-specific evaluation frameworks such as RAGAS emphasize faithfulness, answer relevancy, and context quality for retrieval-augmented systems [14]. Token-level answer comparison can also be reported using QA-style F1 when reference answers are available [15], while semantic similarity metrics such as BERTScore are useful when generated wording differs from the reference answer but preserves meaning [16].

The Phase 2 implementation therefore satisfies the design requirement of evaluating multiple alternative solutions before selecting the final architecture. Dense-only retrieval is currently the strongest retrieval baseline on the single-domain Bangla evaluation set, while CivicRAG remains the selected system design because it is more explainable, supports source-grounded responses, and is better prepared for future multi-domain government-service expansion.

References Used in Chapter 4

1. Lewis, Patrick, Perez, Ethan, Piktus, Aleksandra, Petroni, Fabio, Karpukhin, Vladimir, Goyal, Naman, K{\u. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. *Advances in Neural Information Processing Systems*. 2020. <https://arxiv.org/abs/2005.11401>.
2. Opin, Sayed Ashraful Islam, Thakur, Raiyan Ahmed, Raiyan, Ahsanul Karim, Kabir, Muhammad Rafsan, Rahman, Shafin. “NextRAG: A Bilingual Retrieval-Augmented Generation Framework for

- Financial Question-Answering”. 2025 28th International Conference on Computer and Information Technology (ICCIT). 2025. 10.1109/ICCIT68739.2025.11491522.
3. Robertson, Stephen, Zaragoza, Hugo. “The Probabilistic Relevance Framework: BM25 and Beyond”. Foundations and Trends in Information Retrieval. 2009. 10.1561/15000000019.
 4. Karpukhin, Vladimir, Oguz, Barlas, Min, Sewon, Lewis, Patrick, Wu, Ledell, Edunov, Sergey, Chen, Danqi, Yih, Wen-tau. “Dense Passage Retrieval for Open-Domain Question Answering”. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. 2020. 10.18653/v1/2020.emnlp-main.550.
 5. Reimers, Nils, Gurevych, Iryna. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. 2019. 10.18653/v1/D19-1410.
 6. Chen, Jianlv, Xiao, Shitao, Zhang, Peitian, Luo, Kun, Lian, Defu, Liu, Zheng. “BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation”. 2024. <https://arxiv.org/abs/2402.03216>.
 7. Cormack, Gordon V., Clarke, Charles L. A., Buettcher, Stefan. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods”. Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. 2009. 10.1145/1571941.1572114.
 8. Nogueira, Rodrigo, Cho, Kyunghyun. “Passage Re-ranking with BERT”. 2019. <https://arxiv.org/abs/1901.04085>.
 9. Manning, Christopher D., Raghavan, Prabhakar, Sch{\'u}. “Introduction to Information Retrieval”. 2008. <https://nlp.stanford.edu/IR-book/>.
 10. Voorhees, Ellen M.. “The TREC-8 Question Answering Track Report”. Proceedings of the Eighth Text REtrieval Conference. 1999. https://trec.nist.gov/pubs/trec8/papers/qa_report.pdf.
 11. J{\'a}. “Cumulated Gain-Based Evaluation of IR Techniques”. ACM Transactions on Information Systems. 2002. 10.1145/582415.582418.
 12. Dubey, Abhimanyu, others. “The Llama 3 Herd of Models”. 2024. <https://arxiv.org/abs/2407.21783>.
 13. Yang, An, Yang, Baosong, Zhang, Beichen, Hui, Binyuan, Zheng, Bo, Yu, Bowen, Li, Chengyuan, Liu, Dayiheng, Huang, Fei, Wei, Haoran, others. “Qwen2.5 Technical Report”. arXiv preprint arXiv:2412.15115. 2025. <https://arxiv.org/abs/2412.15115>.
 14. Es, Shahul, James, Jithin, Espinosa-Anke, Luis, Schockaert, Steven. “RAGAS: Automated Evaluation of Retrieval Augmented Generation”. Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations. 2024. 10.18653/v1/2024.eacl-demo.16.
 15. Rajpurkar, Pranav, Zhang, Jian, Lopyrev, Konstantin, Liang, Percy. “SQuAD: 100,000+ Questions for Machine Comprehension of Text”. Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing. 2016. 10.18653/v1/D16-1264.
 16. Zhang, Tianyi, Kishore, Varsha, Wu, Felix, Weinberger, Kilian Q., Artzi, Yoav. “BERTScore: Evaluating Text Generation with BERT”. International Conference on Learning Representations. 2020. <https://openreview.net/forum?id=SkeHuCVFDr>.